

Yesterday 4:02 PM

HW2-2(1).pdf
PDF

Walk me through the homework,

Yep. This homework is basically building a small secure messaging protocol in four stages: first you intentionally show why encryption alone is weak, then you add authenticated key exchange, then a secure record layer, then attack your own implementation with tests. The assignment explicitly wants you to use library implementations of AES-CTR, HMAC-SHA-256, finite-field Diffie–Hellman, RSA-PSS, and SHA-256 rather than implementing the crypto primitives yourself.  HW2-2

I'd do it in this order.

1. **Task 0 — Set up your VM environment.** Inside your course VM, go to your `csce465-agentsec` folder, create/activate the Python virtual environment, and install the exact versions of `cryptography` and `pytest` shown in the assignment. Then generate the `ffdhe3072.pem` Diffie–Hellman parameter file with OpenSSL. Before moving on, save the output of `python3 --version`, `openssl version`, and `pip show cryptography pytest`; that evidence belongs in the Lab Preparation section of your report.  HW2-2
2. **Task 1 — Break AES-CTR on purpose.** Create `baseline_ctr.py`. The plaintext is fixed:

 Plain text

```
{"action": "READ", "path": "notes.txt"}
```

You encrypt it using AES-CTR **without a MAC**. Then you write a relay that changes `READ` to another **same-length** word by modifying ciphertext bytes, without knowing the AES key. You also send the exact same ciphertext twice and show that the receiver accepts/processes it twice. The point is to demonstrate that encryption gives confidentiality, but by itself gives neither integrity nor replay protection.  HW2-2

The key concept for the bit flip is CTR's structure:

 Plain text

```
C = P XOR keystream
```

So if you know the original plaintext `P` and want a desired plaintext `P'`, you can compute:

 Plain text

$C' = C \oplus P \oplus P'$

for just those affected bytes. You never need the AES key.

For your report, Task 1 needs three things in your own words: what you designed, what happened in the experiment, and the root cause.  HW2-2

3. **Task 2 — Build an authenticated Diffie–Hellman handshake.** This is the most complicated part. You simulate a gateway and node locally; no sockets are required. Each side has a long-term 3072-bit RSA signing key. Every new session also creates a fresh ephemeral Diffie–Hellman private value and a fresh 16-byte nonce.  HW2-2

You then construct one **canonical transcript** containing, in this exact conceptual order:

 Plain text

```
protocol label
group identifier
gateway identity
node identity
gateway DH public value
node DH public value
gateway nonce
node nonce
```

The protocol label is `CSCE465-HS-v2` and the group is `ffdhe3072`. Every field must be encoded as:

 Plain text

```
4-byte big-endian length || field bytes
```

You cannot just concatenate the raw fields, because concatenation can be ambiguous. The assignment specifically requires the length-prefixed encoding.  HW2-2

Next calculate:

 Plain text

```
TH = SHA256(transcript)
```

and each party signs:

</> Plain text

```
role || TH
```

using RSA-PSS with SHA-256. Each side verifies the other side's signature and identity before accepting the session.  HW2-2

Then both sides independently calculate the same Diffie–Hellman shared secret Z , encode it as exactly **384 bytes**, and derive the master key:

</> Plain text

```
K_master = SHA256("CSCE465-KDF-v1" || Z || TH)
```

From that you derive four directional keys:

</> Plain text

```
K_g2n_enc
K_g2n_mac
K_n2g_enc
K_n2g_mac
```

plus an 8-byte `session_id`. Use the exact HMAC labels in the PDF; the assignment says not to invent your own KDF.  HW2-2

So mentally, Task 2 is:

</> Plain text

Gateway	Node
generate ephemeral DH	generate ephemeral DH
generate nonce	generate nonce
+----- exchange public data --->+	
+<----- exchange public data ----+	
build same transcript	build same transcript
hash transcript	hash transcript

sign hash + role	sign hash + role
verify peer	verify peer
compute DH shared secret	compute DH shared secret
derive same master key	derive same master key
derive directional keys	derive directional keys

You also need to reject bad signatures, changed nonces/public values, malformed transcripts, wrong identities, and reflected handshake messages. In your report you need to connect this to Diffie–Hellman, signatures, entity authentication, nonces, replay resistance, forward secrecy, and key separation, and explain what becomes possible if the signatures disappear.  HW2-2

4. Task 3 — Build the actual protected message format. Create `secure_record.py` with `seal()` and `open_record()` .

Every message has this authenticated header:

 Plain text

```
version      1 byte
direction    1 byte
sequence     8 bytes
message_type 1 byte
ciphertext_len 4 bytes
```

Then:

 Plain text

```
iv = session_id || sequence
ciphertext = AES-256-CTR(K_enc, iv, plaintext)
tag = HMAC-SHA-256(K_mac, header || iv || ciphertext)
```

 HW2-2

The sender starts its sequence number at `0` . Each direction has its own independent sequence number and its own encryption/MAC keys. The receiver only accepts the exact next expected sequence number. Most importantly, on receive you should conceptually do:

 Plain text

```
parse record
check basic format
check direction
```

```

check sequence
verify HMAC
ONLY THEN decrypt

```

Never decrypt first and then check the MAC. The assignment explicitly requires verifying the HMAC before decrypting and releasing no plaintext after any error.  HW2-2

5. **Task 4 — Attack the completed implementation with pytest.** You need automated tests for at least six cases: a valid handshake/messages case, modified ciphertext, modified header, replayed record, reflected record sent in the opposite direction, and one handshake-authentication failure such as the wrong RSA key or invalid transcript signature. Each test has to assert a specific safe failure.

 HW2-2

A sensible test layout would be something like:

</> Plain text

```

tests/
  test_handshake.py
  test_records.py

```

For example:

</> Python

```

def test_modified_ciphertext_rejected():
    ...
    with pytest.raises(InvalidSignature):
        open_record(...)

```

The exact exception type is your design choice, but the important thing is that the corrupted record gets rejected and no plaintext is returned.

6. **Write the security note.** Task 4 also includes a **500–700 word security note** explaining your attacker model and non-goals, why CTR needs unique IVs, why verify-before-decrypt matters, why encryption and MAC use independent keys, how this compares to an AEAD mode like AES-GCM, and why cryptographically valid data can still contain a dangerous authorized tool call.  HW2-2

Your final folder should roughly look like:

</> Plain text

```

hw2/
├─ report.pdf

```